Sales and Customer Data Exploration/Analysis

Spring 2026 Data Science Project

Group Members: Nicholas Shidle, Katy Wouters, Alexa Tong, Sam Wober, Ethan Zhang

Contributions (For each member, list which of the following sections they worked on, and summarize the contributions in 1-2 sentences. Be specific!)

1. Sam: I worked on Conclusion 3 of the Exploratory data analysis section [C], and did the insights and conclusion aspect of the project [F].
2. Katy: I did the initial data cleaning of our dataset [B] and Conclusion 1 for the data exploration portion [C] where I looked at the difference in sales between electronics, office supplies, and furniture categories. I also helped with the report formatting and some tutorial explanations [G].
3. Ethan: I came up with our dataset for the project [A] and I selected our ML model of K-means clustering and trained it [D][E].
4. Alexa: I worked on Conclusion 3 of the exploratory data analysis section [C], and contributed to the visualizations and analysis of our machine learning model [F].
5. Nick: I worked on Conclusion 2 of the exploritory data analysis section [C], and helped contribute to the data cleaning stage by eliminating null entries [B].

Introduction:

Our topic is sales data classification. We are examining whether customers can be segmented into meaningful groups based on their purchasing behavior. Answering this question matters because it enables retailers to make more targeted decisions in areas such as marketing, advertising, and inventory planning.

Data Curation

The primary dataset used for this tutorial is the [Raw Sales Dataset for RFM & Customer Segmentation].

Source: Charm Myaye Aung via Kaggle.

Description: This dataset contains transaction-level data (Product ID, Category, Quantity, etc.) designed for Recency, Frequency, and Monetary (RFM) analysis.

Preprocessing: Specifically, we took the following steps to clean and organize our dataset.

1. We imported necessary libraries for the project, and we loaded the dataset into a pandas DataFrame.
2. We fixed the data (Customer ID and Location) that was stored in rows rather than columns using ffill().
3. We changed the data types of Quantity, PPU, and Amount categories (using .str.replace(',', '') and pd.to_numeric(...)) since they were all strings with commas instead of floats/integers.
4. We converted the Date strings to datetime objects for statistical analysis.
5. We reordered the DataFrame columns into a logical order.

Tutorial Note: This extensive data cleaning was a critical step to our project. Without addressing these issues, our statistical tests in the following section would have all failed.

For any readers who might need additional help understanding Pandas documentation, you can take a look at the following resource.

```
# Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.tsa.seasonal import STL
from scipy.stats import kruskal
```

```
# Import .csv file as df, make sure to have the .csv in an easily accessible pl
# Also make sure that it is the same file name as below

df = pd.read_csv('raw_rfm_sales_transactions_30000.csv')
```

```
# Data Cleaning

is_headers = df['Product ID'].isnull()
df.loc[is_headers, 'Customer ID'] = df['Transaction ID']
df.loc[is_headers, 'Location'] = df['Date']
df['Customer ID'] = df['Customer ID'].ffill()
df['Location'] = df['Location'].ffill()
```

```
df = df.dropna(subset=['Product ID'])

df['Quantity'] = df['Quantity'].astype(str)
df['Quantity'] = df['Quantity'].str.replace(',', '')
df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
df['Quantity'] = df['Quantity'].fillna(0)
df['Quantity'] = df['Quantity'].astype(int)

df['PPU'] = df['PPU'].astype(str)
df['PPU'] = df['PPU'].str.replace(',', '')
df['PPU'] = pd.to_numeric(df['PPU'], errors='coerce')

df['Amount'] = df['Amount'].astype(str)
df['Amount'] = df['Amount'].str.replace(',', '')
df['Amount'] = pd.to_numeric(df['Amount'], errors='coerce')

df['Date'] = pd.to_datetime(df['Date'], format='%d.%m.%Y', errors='coerce')
order = ['Customer ID', 'Location', 'Transaction ID', 'Date', 'Product ID',
    'Product Name', 'Product Category', 'Quantity', 'PPU', 'Amount']
df = df[order]
df
```

	Customer ID	Location	Transaction ID	Date	Product ID	Product Name	Product Category	Qu
1	Customer-001	Yangon	T000001	2025-01-01	PROD-008	Executive Ballpoint Pen Box	Office Supplies	
2	Customer-001	Yangon	T000002	2025-01-01	PROD-002	Wireless Mechanical Keyboard	Electronics	
3	Customer-001	Yangon	T000003	2025-01-01	PROD-004	Standing Desk Converter	Furniture	
4	Customer-001	Yangon	T000004	2025-01-01	PROD-006	A4 Printer Paper (500 Sheets)	Office Supplies	
5	Customer-001	Yangon	T000308	2025-01-02	PROD-002	Wireless Mechanical Keyboard	Electronics	
...	...	...	...	...	...	...	...	

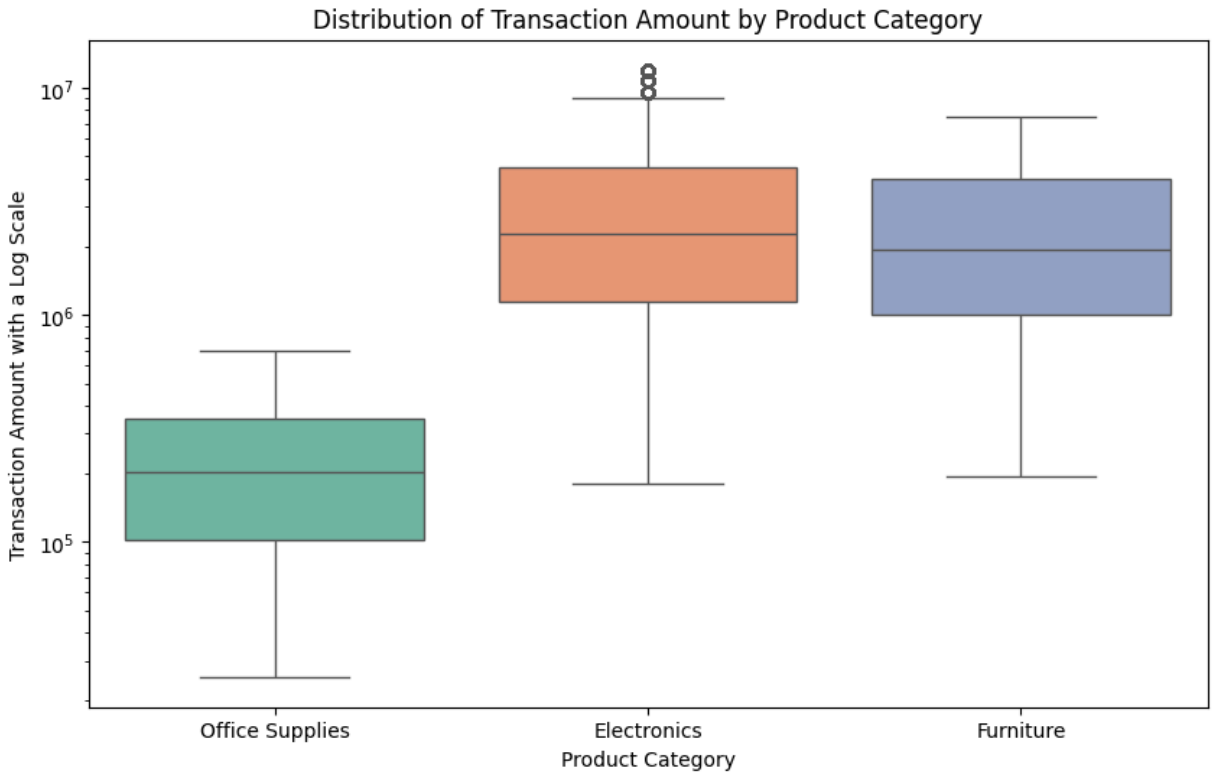
```
# We display the summary statistics to better understand the data

df.describe()
```

	Date	Quantity	PPU	Amount
count	30000	30000.000000	3.000000e+04	3.000000e+04
mean	2025-03-06 06:42:34.560000256	5.490067	4.504362e+05	2.468431e+06
min	2025-01-01 00:00:00	1.000000	2.550000e+04	2.550000e+04
25%	2025-01-25 00:00:00	3.000000	1.800000e+05	5.400000e+05
50%	2025-02-19 00:00:00	5.000000	3.400000e+05	1.520000e+06
75%	2025-04-09 00:00:00	8.000000	7.500000e+05	3.500000e+06
max	2025-06-30 00:00:00	10.000000	1.200000e+06	1.200000e+07
std	NaN	2.880008	3.648356e+05	2.596340e+06

Exploratory Data Analysis

```
# CONCLUSION 1
electronics_revenue = df[df['Product Category'] == 'Electronics']['Amount'].drc
furniture_revenue = df[df['Product Category'] == 'Furniture']['Amount'].dropna(
t_score, p_value = stats.ttest_ind(electronics_revenue, furniture_revenue)
print("T-Score: ", t_score)
print("P-Value: ", p_value)

plt.figure(figsize=(10,6))
sns.boxplot(
    x = 'Product Category',
    y = 'Amount',
    data = df,
    palette = 'Set2',
    hue = 'Product Category',
    legend = False,
)

plt.title('Distribution of Transaction Amount by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Transaction Amount with a Log Scale')

plt.yscale('log')
plt.show()
```

```
T-Score:  17.834550513013774
P-Value:  1.0922152411789663e-70
```



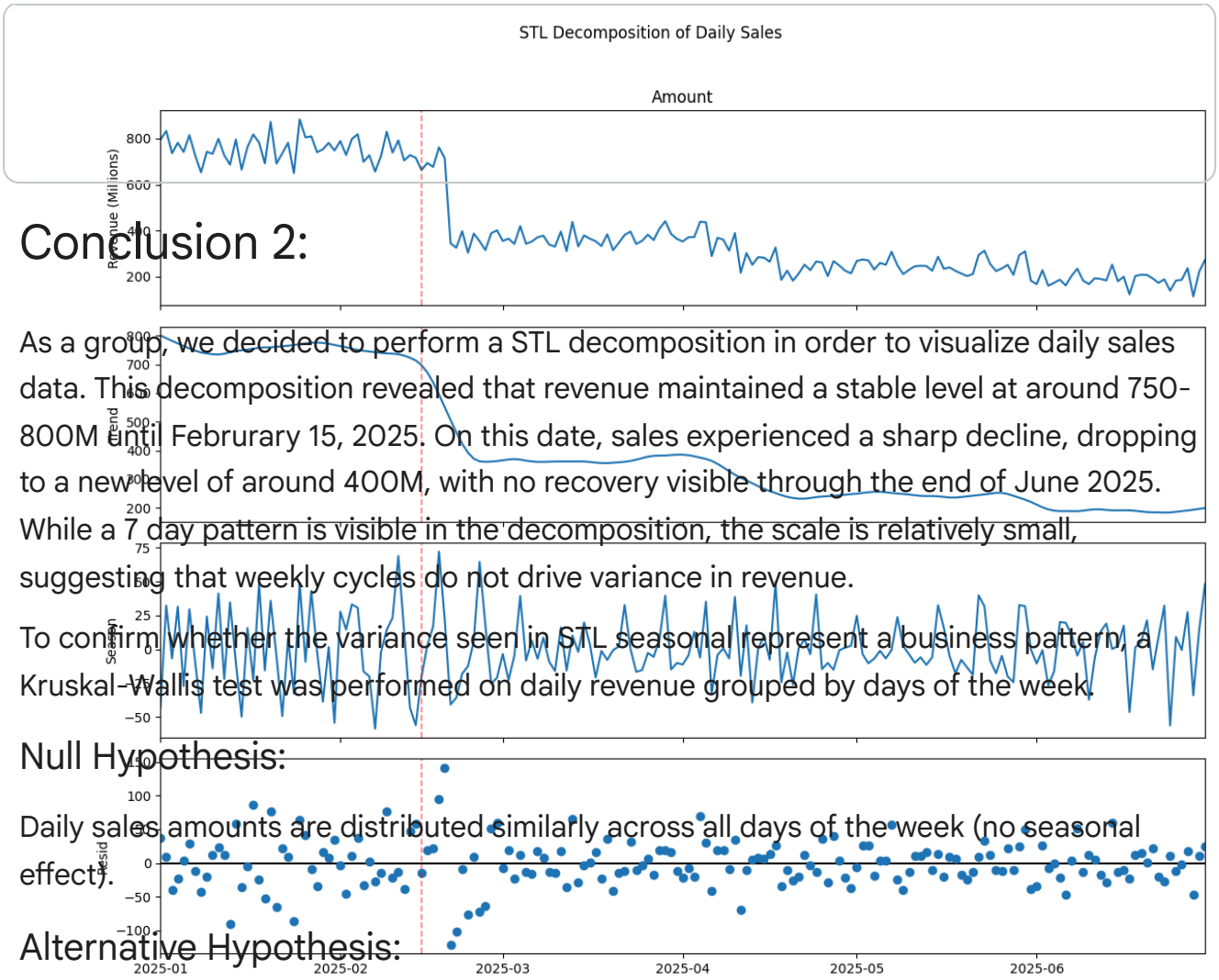
Conclusion 1: Our cleaned dataset has 30,000 transactions with 10 features. In our set, we noticed that there is a massive variance in transaction amount ranging from small amounts to high outliers such as 12,000,000. Thus, we wanted to determine which product category is responsible for a higher transaction amount through an **Independent 2 Sample T-Test**.

H0: There is no significant difference between the mean transaction amount between electronics and furniture. HA: There is a statistically significant difference between the mean transaction amount between electronics and furniture.

The T-Test returned above a T-statistic of 17.83 and a P-value of 1.09x10^-70. Because the p-value falls below the alpha value of 0.05, we **reject the null hypothesis**. Thus, we can draw the conclusion that **electronics generate significantly higher revenue per transaction than furniture**.

To demonstrate this, I generated a box plot with a logarithmic scale to handle outliers. The visualization also upholds this conclusion.

```
# Conclusion 2

# group by day
daily_data = df.groupby('Date')['Amount'].sum().asfreq('D').fillna(0)

# scale data by 1,000,000
daily_data_m = daily_data / 1e6

# stl decomposition and plot
result = STL(daily_data_m, period = 7).fit()
fig = result.plot()
fig.set_size_inches(12, 10)
plt.suptitle('STL Decomposition of Daily Sales', y = 1.02 )

fig.axes[0].set_ylabel('Revenue (Millions)')

# Highlight the Feb drop across all panels
for ax in fig.axes:
    ax.axvline(pd.Timestamp('2025-02-15'), color='red',
               linestyle='--', alpha=0.5, linewidth=1.2)

fig.axes[1].annotate('Revenue decline', xy=(pd.Timestamp('2025-02-15'), 35),
                     xytext=(pd.Timestamp('2025-03-15'), 60),
                     arrowprops=dict(arrowstyle='->', color='red'),
                     fontsize=9, color='red')
plt.tight_layout()
plt.show()

# Kruskal-Wallis test by day of week
df['DayOfWeek'] = pd.to_datetime(df['Date']).dt.day_name()
groups = [grp['Amount'].values for _, grp in df.groupby('DayOfWeek')]
H, p = kruskal(*groups)
print(f"H = {H:.4f}, p = {p:.4f}")
print("Significant seasonal difference!" if p < 0.05 else "No significant diffe
```



Conclusion 2:

As a group, we decided to perform a STL decomposition in order to visualize daily sales data. This decomposition revealed that revenue maintained a stable level at around 750-800M until Februrary 15, 2025. On this date, sales experienced a sharp decline, dropping to a new level of around 400M, with no recovery visible through the end of June 2025. While a 7 day pattern is visible in the decomposition, the scale is relatively small, suggesting that weekly cycles do not drive variance in revenue.

To confirm whether the variance seen in STL seasonal represent a business pattern, a Kruskal-Wallis test was performed on daily revenue grouped by days of the week.

Null Hypothesis:

Daily sales amounts are distributed similarly across all days of the week (no seasonal effect).

Alternative Hypothesis:

At least one day of the week significantly differs in sales volume.

```
H = 10.6264, p = 0.1006
No significant difference.
```

Test Results:

H = 10.6264, p = 0.1006 No significant difference.

Since the p value is greater than the alpha level of 0.05, we fail to reject the null hypothesis. There is no statistically significant evidence of a weekday specific performance bias.

Conclusion 3:

We also want to observe the relationship between purchase frequency and total spending to understand whether more frequent purchases are associated with higher overall spending. This led us to use the Spearman Rank correlation test as it allows us to find the correlation between the frequency of purchases and the total spending while accounting for outliers through the use of ranks.

H_0: There is no positive monotonic correlation between purchase frequency and total spending. Mathematically, $\rho_s \leq 0$
H_A: There is a positive monotonic correlation between purchase frequency and total spending. Mathematically, $\rho_s > 0$

```
# Create customer_df to track frequency of purchases and total spending per cus
customer_df = df.groupby('Customer ID').agg(
  Frequency=('Transaction ID', 'count'),
  Total_Spending=('Amount', 'sum')
).reset_index()
customer_df
```

	Customer ID	Frequency	Total_Spending
0	Customer-001	322	784745000
1	Customer-002	262	679040500
2	Customer-003	287	736283000
3	Customer-004	318	689126000
4	Customer-005	311	765826500
...	...	...	...
95	Customer-096	313	829229500
96	Customer-097	295	735736000
97	Customer-098	305	715626000
98	Customer-099	295	700938500
99	Customer-100	292	741243000

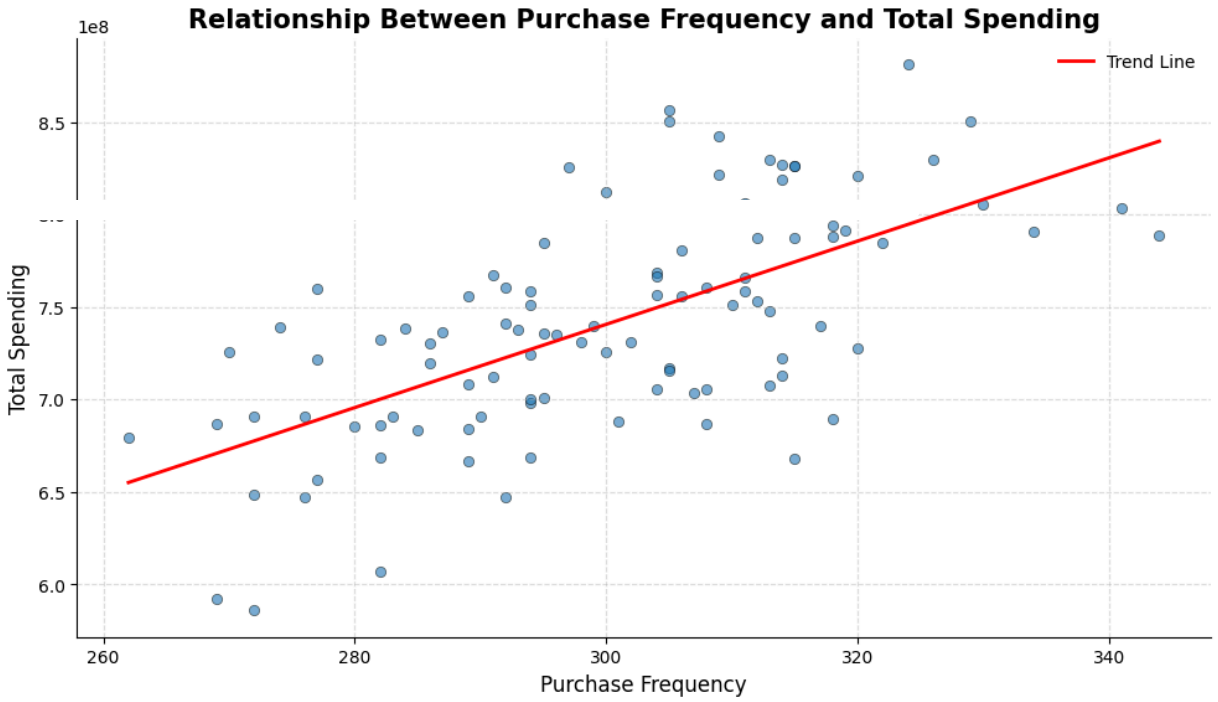
100 rows × 3 columns

```
# Use Spearman rank test
rho, p_value = stats.spearmanr(customer_df['Frequency'], customer_df['Total_Spe
print('p value:', p_value)
print('rho:', rho)
```

```
p value: 7.771735635357558e-13
rho: 0.639757049856652
```

```
# Plot correlation on scatter plot
x = customer_df['Frequency']
y = customer_df['Total_Spending']

plt.figure(figsize=(10,6))
```

```
plt.scatter(x, y, alpha=0.6, edgecolor='k', linewidth=0.5,)

m, b = np.polyfit(x, y, 1)
x_sorted = np.sort(x)
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line'

plt.title('Relationship Between Purchase Frequency and Total Spending', fontsiz
plt.xlabel('Purchase Frequency', fontsize=12)
plt.ylabel('Total Spending', fontsize=12)

plt.grid(True, linestyle='--', alpha=0.4)
plt.legend(frameon=False)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.tight_layout()

plt.show()
```



Let the significance level be $\alpha = 0.05$. The p-value obtained from the Spearman Rank correlation test is 7.77×10^{-13} which is less than α. Therefore, we reject the null hypothesis in support of the alternative hypothesis. We can conclude that there is a statistically significant positive monotonic relationship between purchase frequency and total spending since $\rho_s = 0.6397570498566 52 > 0$. Therefore, as purchase frequency increases, total spending also increases.

ML Algorithm Design: Primary Analysis

Based on our exploratory data analysis, we selected **K-Means Clustering** as our machine learning algorithm to group the customer base. We chose this algorithm in particular because it fits our purpose perfectly allowing us to identify naturally ocurring sectors within the sales data based on their RFM (Recency, Frequency, Monerary) characteristics. Our goal is to use this unsupervised technique to place customers into 4 different categories:

1. Priority (High Frequency, High Spending)
2. Loyal (High Frequency, Lower Spending)
3. Occasional A (Lower Frequency, High Spending)
4. Occasional B (Lower Frequency, Lower Spending)

For advanced readers who would like to learn more about K-Means Clustering, please check out the following resource: https://scikit-learn.org/stable/modules/clustering.html.

ML Algorithm Development

Calculate Recency, Frequency, and Monetary values

```
# Reference date is the last date in the dataset
reference_date = df['Date'].max()

rfm = df.groupby('Customer ID').agg({
    'Date': lambda x: (reference_date - x.max()).days,
    'Transaction ID': 'count',
    'Amount': 'sum'
}).reset_index()

rfm.columns = ['Customer_ID', 'Recency', 'Frequency', 'Monetary']
```

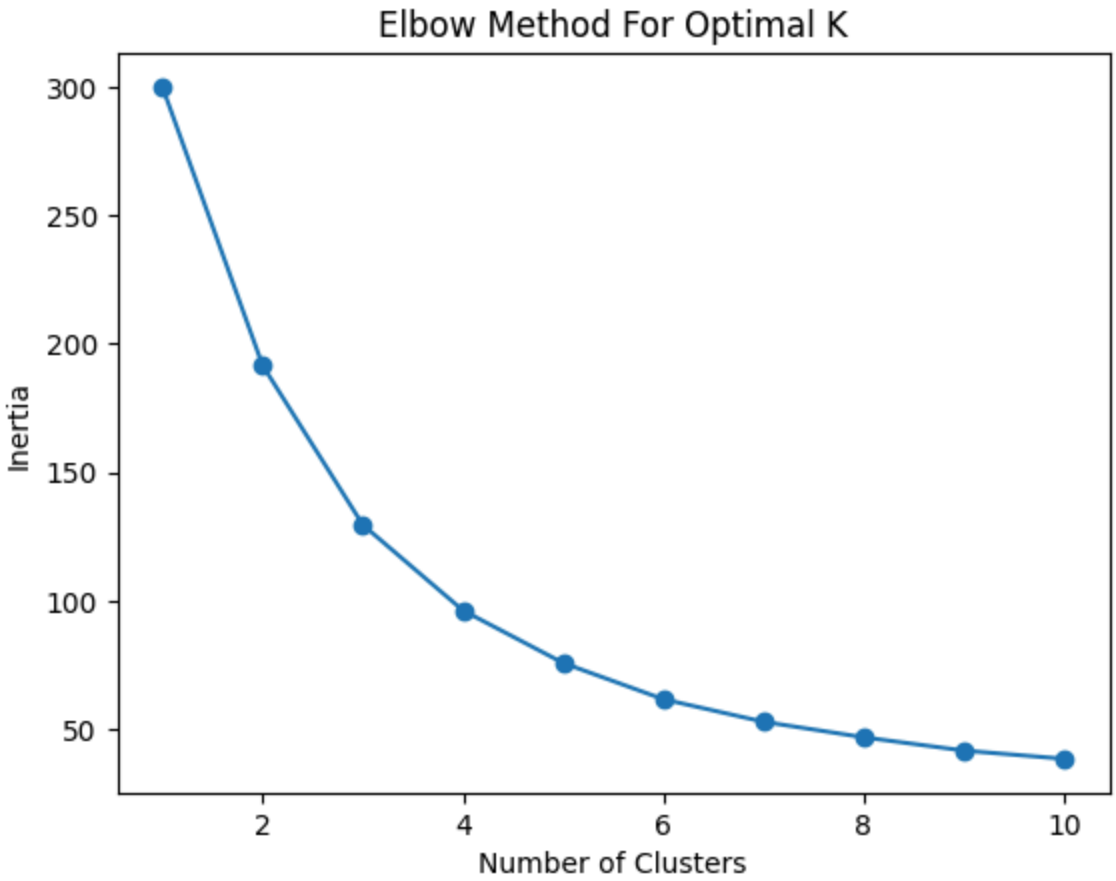
ML Algorithm Training

Log transform and standardize to prevent outliers from skewing clusters

```
from sklearn.preprocessing import StandardScaler

# Log transform
rfm_log = rfm[['Recency', 'Frequency', 'Monetary']].apply(np.log1p, axis=1)

# Standardize
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm_log)
```

Find optimal K using elbow method

```
from sklearn.cluster import KMeans

inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(rfm_scaled)
    inertia.append(kmeans.inertia_)

plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method For Optimal K')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.show()
```



We identified K = 4 as the optimal value since that is where the decrease in inertia slows significantly.

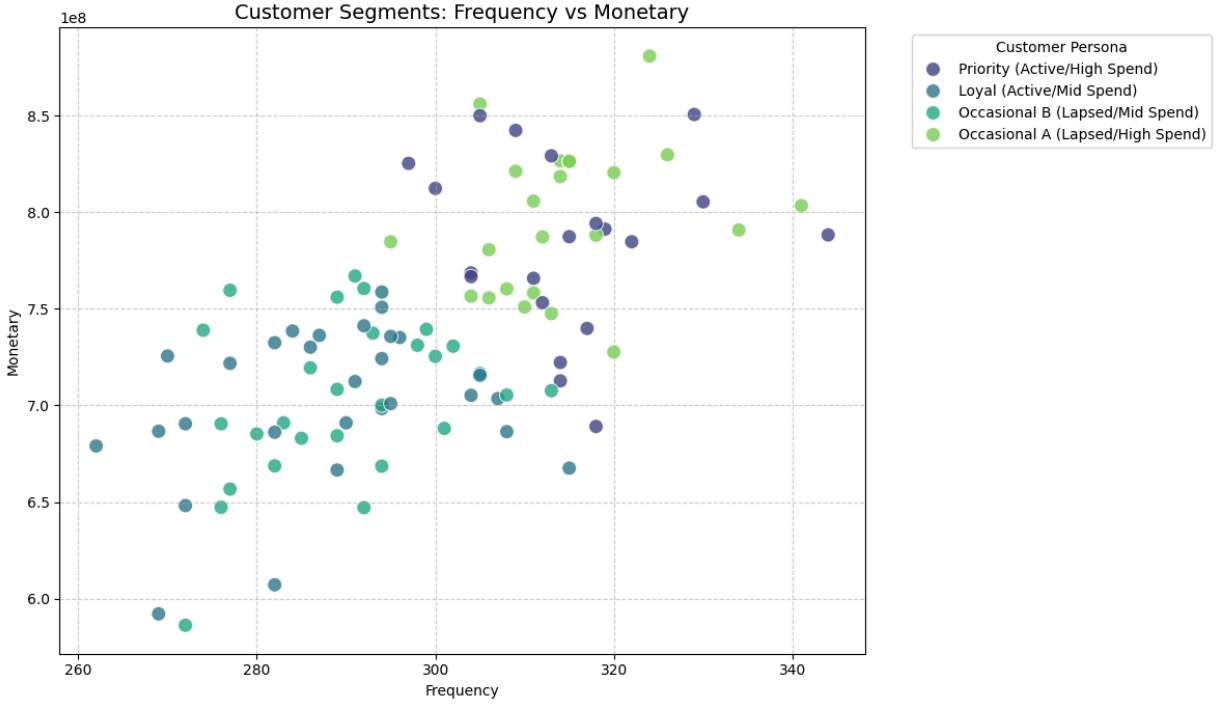
```
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
rfm['Cluster'] = kmeans.fit_predict(rfm_scaled)

# View the avg characteristics of each cluster
cluster_profiles = rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']]
print(cluster_profiles)
```

```
            Recency   Frequency      Monetary
Cluster
0          0.482759  288.172414  7.023553e+08
1         98.857143  289.892857  7.035993e+08
2          0.150000  314.750000  7.839884e+08
3         91.956522  314.391304  7.958294e+08
```

Test Data Analysis

To analyze our algorithm, we used a scatter plot to visualize how our customers are grouped based on their total spending (Monetary) and how often they shop (Frequency).

```
cluster_map = {
    0: 'Loyal (Active/Mid Spend)',
    1: 'Occasional B (Lapsed/Mid Spend)',
    2: 'Priority (Active/High Spend)',
    3: 'Occasional A (Lapsed/High Spend)',
}

rfm['Cluster_Label'] = rfm['Cluster'].map(cluster_map)

# Visualize with the new classifications
plt.figure(figsize=(12, 7))
sns.scatterplot(
    data=rfm,
    x='Frequency',
    y='Monetary',
    hue='Cluster_Label', # Use the string labels here
    palette='viridis',
    s=100,
    alpha=0.8
)

plt.title('Customer Segments: Frequency vs Monetary', fontsize=14)
plt.legend(title='Customer Persona', bbox_to_anchor=(1.05, 1), loc='upper left'
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# Segment Profile visualization
cluster_summary = rfm.groupby('Cluster_Label').agg({
    'Recency': 'mean',
    'Frequency': 'mean',
    'Monetary': ['mean', 'count']
}).round(2)
print(cluster_summary)
```



Cluster_Label	Recency mean	Frequency mean	Monetary mean	count
Loyal (Active/Mid Spend)	0.48	288.17	7.023553e+08	29
Occasional A (Lapsed/High Spend)	91.96	314.39	7.958294e+08	23
Occasional B (Lapsed/Mid Spend)	98.86	289.89	7.035993e+08	28
Priority (Active/High Spend)	0.15	314.75	7.839884e+08	20

Visualization

Our visualization is a silhouette plot to represent the four distinct customer personas discovered by our clustering algorithm. A Silhouette Score is a measurement of how

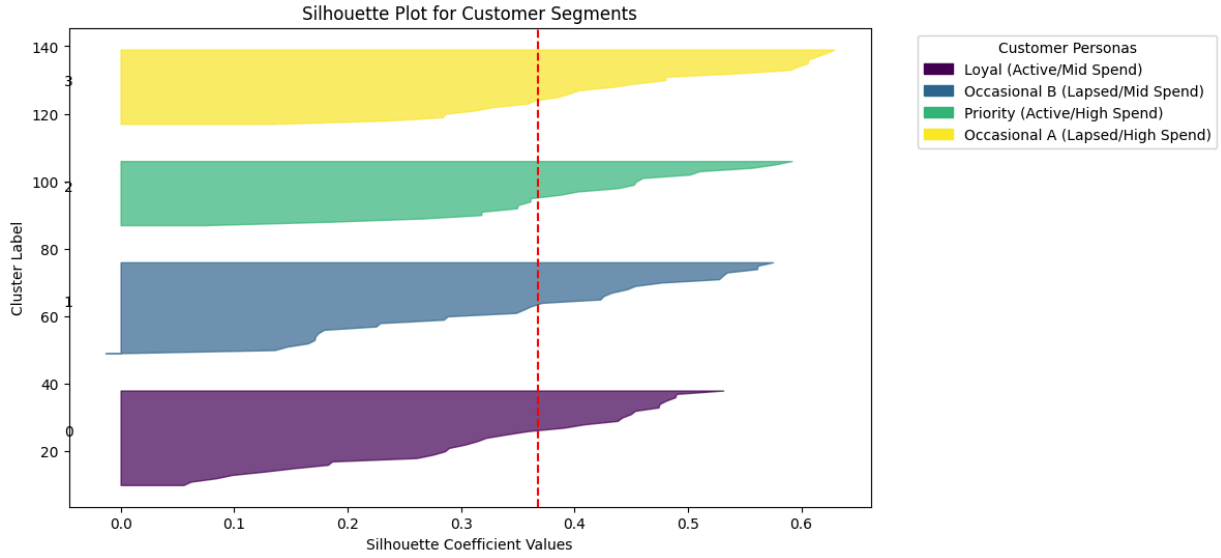
similar a data point is to its own cluster compared to other clusters, ranging from -1 to 1. A score of 1 means the clusters are perfectly separated, while a score near 0 means they are overlapping. In our test data analysis, we obtained a silhouette score of 0.368 indicating a certain level of separation between the categories with some overlapping. This proves the segments are distinct enough for business use and marketing strategies.

To dive deeper into the mathematics behind unsupervised cluster validation, we recommend any interested readers to take a look at the Scikit-Learn guide on Silhouette Analysis

```
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.cm as cm
import matplotlib.patches as mpatches

# Calculate the avg score
avg_score = silhouette_score(rfm_scaled, rfm['Cluster'])
print(f"The average silhouette_score is: {avg_score:.3f}")

cluster_map = {
    0: 'Loyal (Active/Mid Spend)',
    1: 'Occasional B (Lapsed/Mid Spend)',
    2: 'Priority (Active/High Spend)',
    3: 'Occasional A (Lapsed/High Spend)'
}

# Compute silhouette scores for each sample
sample_silhouette_values = silhouette_samples(rfm_scaled, rfm['Cluster'])

fig, ax1 = plt.subplots(figsize=(10, 6))
y_lower = 10

colors = cm.viridis(np.linspace(0, 1, 4))
legend_patches = []

for i in range(4): # For the 4 clusters
    # Aggregate and sort scores for each cluster
    ith_cluster_silhouette_values = sample_silhouette_values[rfm['Cluster'] ==
    ith_cluster_silhouette_values.sort()

    size_cluster_i = ith_cluster_silhouette_values.shape[0]
    y_upper = y_lower + size_cluster_i

    color = colors[i]
    ax1.fill_betweenx(np.arange(y_lower, y_upper), 0, ith_cluster_silhouette_va
                      facecolor=color, edgecolor=color, alpha=0.7)

    ax1.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
    legend_patches.append(mpatches.Patch(color=color, label=cluster_map[i]))
```

```
    y_lower = y_upper + 10

ax1.axvline(x=avg_score, color="red", linestyle="--")
ax1.set_title("Silhouette Plot for Customer Segments")
ax1.set_xlabel("Silhouette Coefficient Values")
ax1.set_ylabel("Cluster Label")
plt.legend(handles=legend_patches, title="Customer Personas", bbox_to_anchor=(1
plt.show()
```

```
The average silhouette_score is: 0.368
```



The silhouette score of 0.368 suggests some overlap within the clusters. This score indicates that there is substantiable evidence that the categories are distinct, and this information can be applied to real-world business strategies.

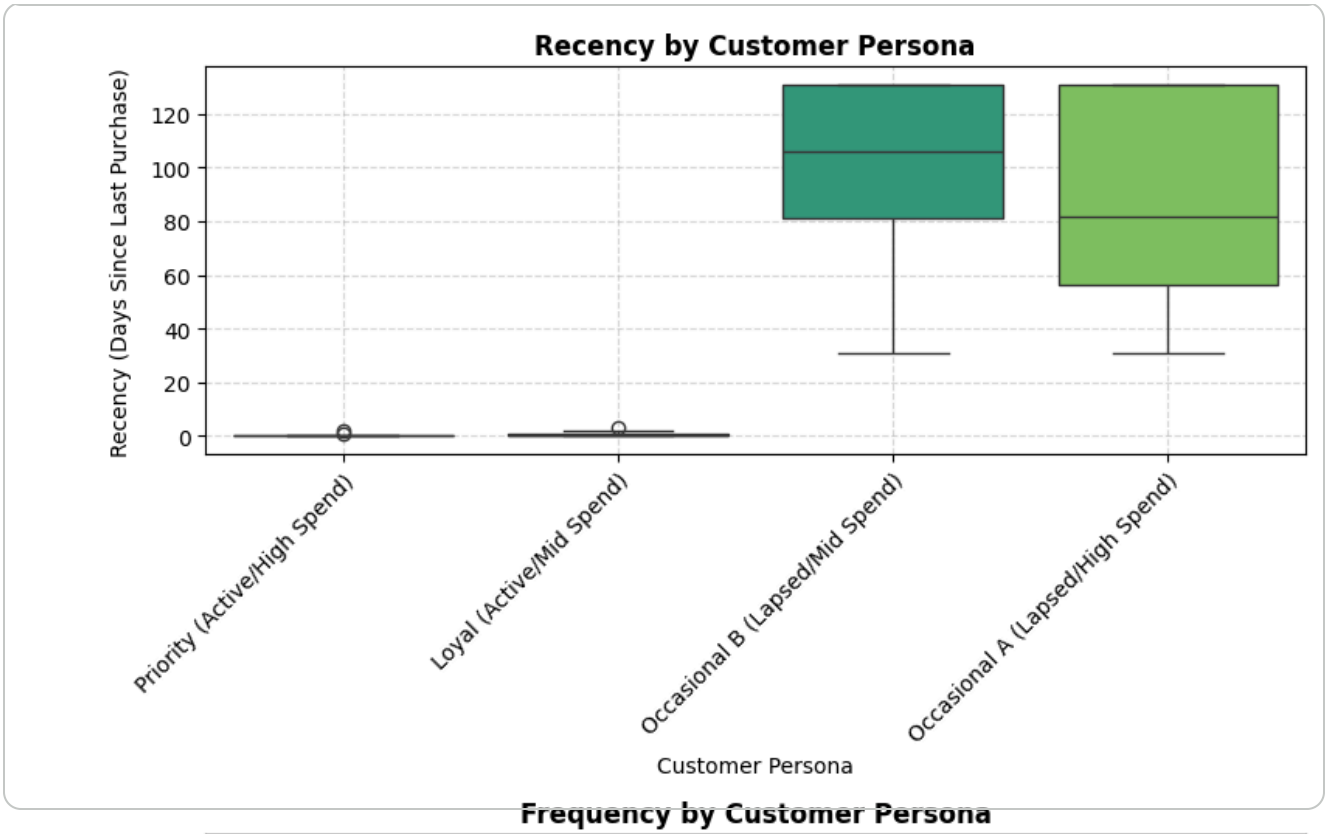
As the legend shows, each color of the plot corresponds to a different segment: Loyal, Priority, and Occasional A/B. Additionally, in our silhouette plot, the greater the width of the segment corresponds to a more predictable and consistent group.

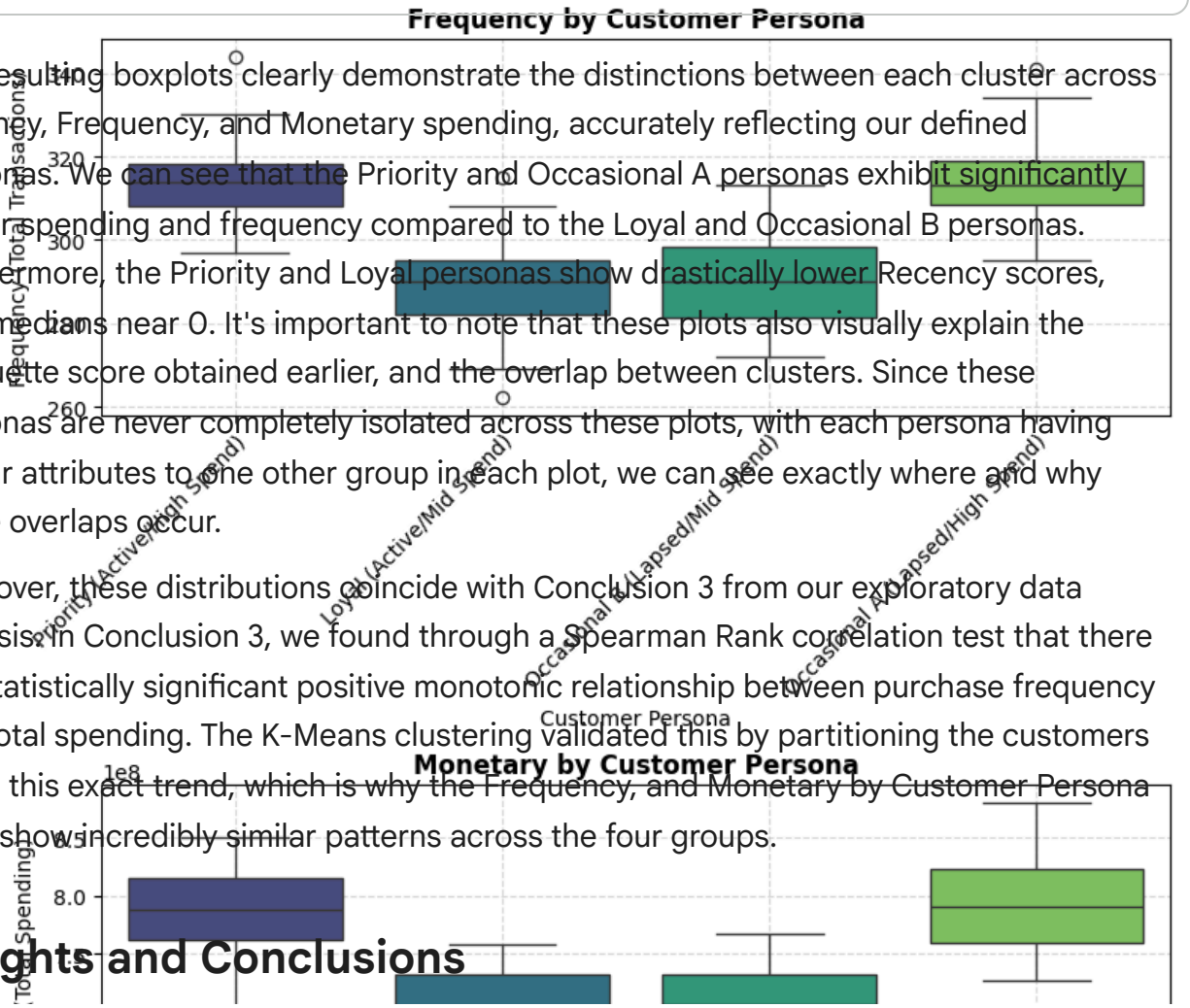
The structure of the silhouette plot shows that none of the clusters dip below 0.0, meaning that almost no customers were misclassified. The clusters are also similar in thickness, meaning that the 100 customers are relatively distributed evenly among the 4 clusters.

Distributions of RFM Across Customer Personas

To further understand and visualize the results of our K-Means clustering, we plotted the distributions of Recency, Frequency, and Monetary spending across our four clusters. This allows us to view the patterns and specific overlaps of the clusters with the original, unstandardized data.

```
# Plot boxplots for Recency, Frequency, and Monetary values
rfm['Customer Persona'] = rfm['Cluster'].map(cluster_map)

fig, axes = plt.subplots(3, 1, figsize=(8, 15))

# Recency plot
sns.boxplot(ax=axes[0], x='Customer Persona', y='Recency', data=rfm, palette='v
axes[0].set_title('Recency by Customer Persona', fontweight='bold')
axes[0].set_ylabel('Recency (Days Since Last Purchase)', labelpad=8)

# Frequency plot
sns.boxplot(ax=axes[1], x='Customer Persona', y='Frequency', data=rfm, palette=
axes[1].set_title('Frequency by Customer Persona', fontweight='bold')
axes[1].set_ylabel('Frequency (Total Transactions)', labelpad=8)

# Monetary plot
sns.boxplot(ax=axes[2], x='Customer Persona', y='Monetary', data=rfm, palette='
axes[2].set_title('Monetary by Customer Persona', fontweight='bold')
axes[2].set_ylabel('Monetary (Total Spending)', labelpad=8)

for ax in axes:
  ax.grid(True, linestyle='--', alpha=0.5)
  ax.set_xticks(ax.get_xticks())
  ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right', rotation_mo

plt.tight_layout()
plt.show()
```





The resulting boxplots clearly demonstrate the distinctions between each cluster across Recency, Frequency, and Monetary spending, accurately reflecting our defined personas. We can see that the Priority and Occasional A personas exhibit significantly higher spending and frequency compared to the Loyal and Occasional B personas. Furthermore, the Priority and Loyal personas show drastically lower Recency scores, with medians near 0. It's important to note that these plots also visually explain the silhouette score obtained earlier, and the overlap between clusters. Since these personas are never completely isolated across these plots, with each persona having similar attributes to one other group in each plot, we can see exactly where and why these overlaps occur.

Moreover, these distributions coincide with Conclusion 3 from our exploratory data analysis. In Conclusion 3, we found through a Spearman Rank correlation test that there is a statistically significant positive monotonic relationship between purchase frequency and total spending. The K-Means clustering validated this by partitioning the customers along this exact trend, which is why the Frequency, and Monetary by Customer Persona plots show incredibly similar patterns across the four groups.

Insights and Conclusions